

Bài tập buổi 8

Cách làm bài

Các file bài tập đều được thêm ở thư mục:

`tests/students-submission/<name>/lesson-8.`

Tổng hợp kiến thức

Làm bài tập tại nhánh `feat/lesson-8-key-takeaways-${name}` và push bài tập lên.

Tạo file `key-takeaways.md`, tổng hợp lại các kiến thức được học trong bài.

Git

Kiến thức bổ sung để làm bài: đặt nhánh mặc định

Mặc định, tại một số hệ thống git cũ, khi bạn chạy lệnh: `git init` thì nhánh mặc định sẽ là “master”.

Hiện nay, khái niệm “master-slave” (ông chủ và nô lệ) gây tranh cãi, nên người ta có xu hướng chuyển sang nhánh “main”.

Để cài đặt mặc định, ta sử dụng lệnh sau:

```
git config --global init.defaultBranch main
```

Hãy tiến hành chạy trên máy của bạn luôn nhé ^^. Từ bài này, chúng ta sẽ sử dụng nhánh “main”

Assertion & Web first assertion

Dùng để kiểm tra một giá trị có đúng với mong muốn không.

Ví dụ: kiểm tra 1 phần tử có chứa text “Playwright class” hay không

```
<p> Playwright class from Playwright Việt Nam </p>
```

```
const elem = page.locator('//p')
```

```
await expect(elem).toContainText("Playwright class");
```

Ở bài này, chúng ta sẽ sử dụng các async matcher (tức là sẽ luôn đi kèm `await`) ở bảng dưới đây:

Web first assertion	Ý nghĩa
<code>await expect(elem).toBeAttached();</code>	Kiểm tra phần tử đã được gắn vào DOM
<code>await expect(elem).toBeChecked();</code>	Kiểm tra phần tử đã được check. Thường dùng cho checkbox, radio button.
<code>await expect(elem).toBeEditable();</code>	Kiểm tra phần tử có thể sửa được. Thường dùng cho ô input
<code>await expect(elem).toBeEmpty();</code>	Kiểm tra phần tử rỗng. Thường dùng cho các phần tử warning, error.
<code>await expect(elem).toBeEnabled();</code>	Kiểm tra phần tử có được enable hay không. Thường dùng cho button hoặc input
<code>await expect(elem).toBeFocused();</code>	Kiểm tra phần tử có được focus hay không. Thường dùng cho input
<code>await expect(elem).toBeHidden();</code>	Kiểm tra phần tử có bị ẩn khỏi trang web hay không. Thường dùng cho các text thông báo
<code>await expect(elem).toBeInViewport();</code>	Kiểm tra phần tử có nằm trong viewport hay không.
<code>await expect(elem).toBeVisible();</code>	Kiểm tra phần tử có visible (hiển thị) hay không
<code>await expect(elem).toContainText("abc");</code>	Kiểm tra phần tử có chứa text hay không
<code>await expect(elem).toHaveAttribute("href");</code>	Kiểm tra phần tử có thuộc tính hay không
<code>await expect(elem).toHaveClass("class-name");</code>	Kiểm tra phần tử có class hay không
<code>await expect(elem).toHaveId("id")</code>	Kiểm tra phần tử có id hay không
<code>await expect(elem).toHaveText('');</code>	Kiểm tra phần tử có text hay không
<code>await expect(elem).toHaveValue('')</code>	Kiểm tra input có chứa giá trị hay không

<code>await expect(elem).toHaveValues([])</code>	Kiểm tra select có select các option hay không
--	--

Assertion	Ý nghĩa
<code>expect().toBe();</code>	Kiểm tra phần tử đã được gắn vào DOM
<code>expect().toEqual();</code>	Kiểm tra hai giá trị bằng nhau (so sánh sâu cho object/array)
<code>expect().toContain();</code>	Kiểm tra một phần tử có trong array hoặc chuỗi
<code>expect().toBeTruthy();</code>	Kiểm tra giá trị có "truthy" (không null, không false, không undefined)
<code>expect().toBeFalsy();</code>	Kiểm tra giá trị có "falsy" (null, undefined, 0, hoặc false)
<code>expect().toBeGreaterThan();</code>	Kiểm tra giá trị lớn hơn
<code>expect().toBeLessThan();</code>	Kiểm tra giá trị nhỏ hơn

Bài tập

Làm bài tập tại nhánh `feat/lesson-8-git-${name}` và push bài tập lên

1. Tạo file `01-git-answer.txt` trong thư mục để trả lời câu hỏi sau: giải thích câu lệnh `git commit --amend, git reset HEAD~3`
2. Liệt kê các commit và các file theo từng vùng ở các branch: main, feature-A, feature-B sau khi thao tác các lệnh sau:
 - a. Tạo thư mục `lesson-8`
 - b. Tạo file: `file1, file2, file3`
 - c. Chạy lệnh: `git init`
 - d. Chạy lệnh: `git add file1`
 - e. Chạy lệnh: `git commit -m"init project"`
 - f. Chạy lệnh: `git checkout -b feat/feature-A`
 - g. Chạy lệnh: `git add file2`
 - h. Chạy lệnh: `git commit -m"feat: add file2"`

- i. Chạy lệnh: `git add file3`
- j. Chạy lệnh: `git commit -m"feat: add file3"`
- k. Chạy lệnh: `git checkout -b feat/feature-B`
- l. Chạy lệnh: `git reset HEAD~2`
- m. Chạy lệnh: `git stash`
- n. Chạy lệnh: `git checkout main`
- o. Chạy lệnh: `git stash pop`

Format câu trả lời:

- branch: main
 - Các commit:
 - "init project"
 - "feat: add file1"
 - ...
 - Các file theo từng vùng:
 - Vùng working directory:
 - Vùng staging:
 - Vùng repository:
- branch: feature-A
 - ...

Playwright

Tạo branch `feat/lesson-8-${name}` (checkout từ main) để làm bài.

Cho file test case:

https://docs.google.com/spreadsheets/d/1rNfjqdSPshO602I_hi4uWWnZkMshaZzBhgxpvrReF4MU/edit?usp=sharing

Biết cấu trúc file test case sẽ như sau:

- Mỗi module sẽ viết vào một file
- Tên file: `${module-name}.spec.ts`
- Tên group (describe): `${module-code} - ${module-name}`
- Tên test: `${case-code}: ${case name}`

Hãy tiến hành viết code automation cho tất cả các test trong file. Lưu ý sử dụng hooks, step cho đúng.